# Chameleon: A Polymorphic UDP Tunneling Protocol with Session-Unique Wire Formats

william-aqn
ORCID: 0009-0005-4749-1606
z@x-crm.in
https://github.com/william-aqn/genome

April 2026

**DOI:** 10.5281/zenodo.19493628 **License:** CC BY 4.0
**Source code:** https://github.com/william-aqn/genome **Software license:** GPL-3.0

**Abstract**

Deep packet inspection (DPI) systems identify tunneling protocols by matching fixed byte patterns, header structures, and handshake sequences. We present Chameleon, a polymorphic UDP tunneling protocol where each session deterministically derives a unique wire format—called a *genome*—from a pre-shared key (PSK) via HKDF-SHA256. The derived genome controls the packet magic byte, header field ordering, number and size of decoy fields, nonce length, payload length encoding scheme, padding range, and (when stream mode is enabled) per-packet *wagon* size range. A deterministic PRNG based on SHA-256 in counter mode ensures that both endpoints independently derive an identical genome without any handshake.

Wire output achieves approximately 7.9 bits per byte of Shannon entropy, making individual packets statistically indistinguishable from uniform random data. The protocol provides authenticated encryption via ChaCha20-Poly1305, XChaCha20-Poly1305, or AES-256-GCM, with epoch-based nonce construction and a 256-epoch anti-replay sliding window.

On top of the encrypted polymorphic transport, a stream multiplexer provides reliable, ordered delivery with selective acknowledgment (SACK), Jacobson/Karels RTT estimation, NewReno congestion control, and per-stream flow control. An optional *stream mode* emits a constant flow of genome-shaped wagons at a configurable rate envelope, carrying real data when available and random chaff otherwise, defending against passive timing and activity inference on top of structural polymorphism. Empirical evaluation over a transatlantic link shows sustained throughput of 30 Mbit/s with 67 concurrent streams; stream mode preserves 86–97% of direct throughput at its default 500 KB/s–3 MB/s envelope.

Chameleon is implemented in Go with a single external dependency (`golang.org/x/crypto`) and is available as open-source software under the GPL-3.0 license.

**Keywords:** protocol polymorphism, traffic obfuscation, UDP tunneling, deep packet inspection evasion, AEAD encryption, stream multiplexing, cover traffic, traffic analysis resistance, censorship circumvention

## 1 Introduction

Network censorship increasingly relies on deep packet inspection (DPI) to identify and block circumvention tools. Modern DPI systems operate at line rate, matching protocol fingerprints

against databases of known signatures. When a tunneling protocol uses fixed magic bytes, predictable header structures, or recognizable handshake patterns, these become reliable signatures for blocking.

Existing circumvention tools adopt two broad strategies. *Protocol mimicry* disguises tunnel traffic as a permitted protocol (e.g., TLS, HTTP). While effective against simple blacklists, mimicry is fragile: even subtle deviations from the legitimate protocol's state machine, timing, or statistical properties can be detected by sophisticated DPI [17]. *Randomization* approaches, such as obfs4 [1] and ScrambleSuit [18], aim to produce uniformly random output. However, these tools still have fixed elements in their handshake or session establishment phase that can serve as fingerprints.

Chameleon takes a different approach: *per-session polymorphism* of the entire wire format. Rather than randomizing payload bytes while keeping frame structure fixed, Chameleon derives a unique packet format for each session from a pre-shared key. The format itself—field order, field count, encoding schemes, and padding parameters—changes with every session. An observer who does not possess the PSK cannot predict the structure of any given packet, eliminating structural fingerprints entirely.

**Contributions.** This paper makes the following contributions:

1. A *genome derivation* mechanism that deterministically generates a complete wire format specification from a PSK using HKDF and a platform-stable PRNG (section 4).
2. A *layered architecture* that cleanly separates polymorphic framing, authenticated encryption, reliable delivery, and stream multiplexing (section 3).
3. An *epoch-based nonce* construction with anti-replay protection that eliminates nonce-reuse risks without per-packet random nonce generation (section 5).
4. A reliable stream multiplexer over UDP with SACK, NewReno congestion control, and per-stream flow control (section 6).
5. A *stream mode* (section 7) that emits a constant, genome-shaped flow of "wagons" carrying real data or chaff. Per-packet wagon sizes are drawn from a genome-derived range $[W_{\min}, W_{\max}]$; the flow rate drifts inside an operator-configured envelope; burst mode drains the queue "as is" when real data exceeds the envelope.
6. An *empirical evaluation* demonstrating 30 Mbit/s throughput, 67 concurrent streams, wire entropy of 7.9 bits/byte, and 86–97% goodput retention under stream mode (section 9).

The rest of this paper is organized as follows. section 2 surveys related work. section 3 describes the system architecture. section 4 details the genome derivation mechanism. section 5 covers the cryptographic layer. section 6 describes the stream multiplexer. section 7 introduces the stream (cover-traffic) mode. section 8 presents the security analysis. section 9 reports performance results. section 10 outlines future work, and section 11 concludes.

## 2 Related Work

**Tor pluggable transports.** The Tor Project introduced the pluggable transport framework to decouple transport obfuscation from the Tor protocol itself [5]. Notable transports include:

- **obfs4** [1]: Uses a Curve25519 handshake with Elligator 2 encoding [3] to make the initial key exchange indistinguishable from random. Post-handshake, obfs4 applies NaCl secretbox encryption. However, the handshake phase has a fixed two-message structure that sophisticated DPI can detect through timing and length analysis.
- **ScrambleSuit** [18]: A predecessor to obfs4 that introduced polymorphic padding and session ticket authentication. ScrambleSuit's wire format is fixed once the session is established.
- **meek and Snowflake**: Domain-fronting and WebRTC-based transports that piggyback on legitimate cloud infrastructure. These are orthogonal to wire-format polymorphism

and face distinct challenges (cost, latency, provider cooperation).

**Shadowsocks and derivatives.** Shadowsocks [15] encrypts SOCKS5 traffic with stream ciphers (later AEAD). VMess [16] adds a request/response structure and per-session key negotiation. While these tools encrypt payload, they retain fixed structural elements: Shadowsocks uses a deterministic IV-address prefix, and VMess has a recognizable command structure. Recent DPI systems have exploited these regularities.

**Protocol mimicry.** Tools like FTE, Stegotorus, and CensorSpoofer [17] disguise traffic as HTTP, DNS, or other protocols. Mimicry is fundamentally limited by the difficulty of perfectly reproducing a complex protocol's state machine, timing characteristics, and semantic behavior.

**Distinction of Chameleon.** Unlike obfs4 and ScrambleSuit, which have a fixed post-handshake wire format, Chameleon varies the *entire* packet structure per session—including field order, field count, and encoding schemes. Unlike Shadowsocks/VMess, there is no fixed initial structure. Unlike protocol mimicry, Chameleon does not attempt to impersonate any specific protocol, instead producing uniformly random output. The zero-handshake design (both sides derive the genome independently from the PSK) eliminates the handshake-phase fingerprint that affects obfs4.

## 3 System Architecture

Chameleon is structured as a layered protocol stack. Each layer has a single responsibility and communicates with adjacent layers through well-defined interfaces.

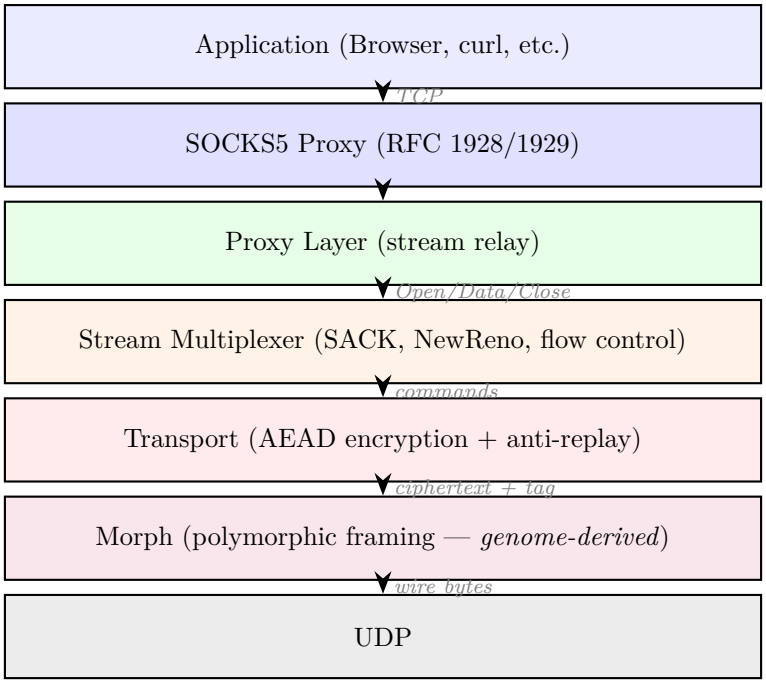


Figure 1: Layered architecture of Chameleon. The morph layer derives its framing rules from the session genome, making the wire format unique per session.

**Design principles.**

1. **Crypto-agnostic morph layer.** The morph layer handles frame serialization and deserialization but never invokes AEAD operations directly. The transport layer orchestrates both encryption and framing. This separation allows each layer to be tested independently.
2. **Zero-handshake session establishment.** Both endpoints derive identical session keys and genome from the shared PSK without any key-exchange messages. This eliminates handshake-phase fingerprints and reduces latency.
3. **Minimal dependencies.** The entire implementation depends only on `golang.org/x/crypto` for ChaCha20-Poly1305 and HKDF. All other functionality uses Go's standard library.

# 4 Genome Derivation

The core novelty of CHAMELEON is the *genome*—a complete specification of the wire format that is deterministically derived from the PSK. Two endpoints sharing the same PSK independently compute identical genomes, enabling communication without any negotiation.

## 4.1 Key Hierarchy

Session keys are derived from the PSK using HKDF-SHA256 [10]:

$$\text{PRK} = \text{HKDF-Extract}(\text{salt}, \text{PSK}) \tag{1}$$

Three values are expanded from the PRK using distinct info strings:

$$K_{\text{AEAD}} = \text{HKDF-Expand}(\text{PRK},\ \texttt{"chameleon-aead-key"},\ 32) \tag{2}$$
$$S_{\text{genome}} = \text{HKDF-Expand}(\text{PRK},\ \texttt{"chameleon-genome-seed"},\ 32) \tag{3}$$
$$B_{\text{nonce}} = \text{HKDF-Expand}(\text{PRK},\ \texttt{"chameleon-nonce-base"},\ 24) \tag{4}$$

where $K_{\text{AEAD}}$ is the 256-bit encryption key, $S_{\text{genome}}$ seeds the genome derivation, and $B_{\text{nonce}}$ provides the fixed prefix for nonce construction. The PSK must be at least 128 bits (16 bytes).

## 4.2 Deterministic PRNG

Genome derivation requires a PRNG that is deterministic and platform-stable: the same seed must produce identical output regardless of Go version, operating system, or CPU architecture. The standard library's `math/rand` does not guarantee algorithm stability across versions.

CHAMELEON uses SHA-256 in counter mode:

$$\text{block}_i = \text{SHA-256}(\text{seed} \parallel \text{counter}_i) \tag{5}$$

where seed $= \text{SHA-256}(S_{\text{genome}})$ is the normalized 256-bit seed and $\text{counter}_i$ is a 64-bit little-endian counter starting at zero. Each invocation produces 32 bytes of pseudorandom output. The PRNG exposes the following operations used during derivation:

- `Bytes`$(n)$: returns $n$ pseudorandom bytes.
- `Intn`$(n)$: returns a uniform integer in $[0, n)$ using rejection sampling to eliminate modulo bias.
- `Shuffle`$(n, \text{swap})$: Fisher-Yates shuffle [7].

## 4.3 Genome Parameters

Given the PRNG seeded with $S_{\text{genome}}$, the genome $G$ is derived as follows:

1. **Magic byte** ($m$): A single pseudorandom byte $m \in [0, 255]$ placed at position 0 of every packet. Acts as a fast-path filter: packets with a wrong magic byte are discarded without further processing.

2. **Nonce size** ($n_s$): Either 12 or 24 bytes, selected uniformly. This determines the AEAD variant: 12-byte nonces use ChaCha20-Poly1305 or AES-256-GCM; 24-byte nonces use XChaCha20-Poly1305.

3. **Length encoding** ($\ell$): One of four schemes, selected uniformly:
   - Big-endian `uint16` (2 bytes, fixed)
   - Little-endian `uint16` (2 bytes, fixed)
   - Unsigned varint (1–3 bytes, variable)
   - XOR-masked big-endian `uint16` with a genome-derived 16-bit mask $\mu$ (2 bytes, fixed)

4. **Core fields**: Five mandatory fields are always present:
   - `Nonce`: $n_s$ bytes
   - `Length`: encoded payload length (scheme-dependent)
   - `PadLen`: 1 byte (padding length)
   - `Tag`: 16 bytes (AEAD authentication tag)
   - `Epoch`: 4 bytes (anti-replay counter, big-endian)

5. **Decoy fields** ($d$): Between 0 and 3 additional fields, each with a random size in $[1, 8]$ bytes. Each decoy has a genome-derived XOR mask; on the wire, decoy bytes are random data XORed with the mask. Decoys are indistinguishable from real fields without knowledge of the genome.

6. **Field order**: All $5 + d$ fields are shuffled using Fisher-Yates with the genome PRNG, producing a permutation $\pi$.

7. **Padding range** ($p_{\min}, p_{\max}$): $p_{\min} \in [0, 15]$, $p_{\max} \in [p_{\min} + 1,\ p_{\min} + 64]$. Each packet is padded with a random number of bytes drawn uniformly from $[p_{\min}, p_{\max}]$ using `crypto/rand`.

8. **Wagon size range** ($W_{\min}, W_{\max}$): $W_{\min} = \mathrm{MSS} + 16 + r_1$ with $r_1 \in [0, 15]$, and $W_{\max} = W_{\min} + 24 + r_2$ with $r_2 \in [0, 79]$. Used only when stream mode is enabled (section 7); both endpoints derive the same range, so they agree on the envelope without negotiation. The constraint $W_{\min} \geq \mathrm{MSS} + 2$ ensures that any mux-layer command fits in the shortest wagon with the 2-byte real-length prefix.

Table 1: Genome parameters and their derivation.

| **Parameter** | **Range** | **Derivation** | **Purpose** |
|---|---|---|---|
| Magic byte $m$ | $[0, 255]$ | `Bytes(1)` | Packet identification |
| Nonce size $n_s$ | $\{12, 24\}$ | `Intn(2)` | AEAD variant selection |
| Length encoding $\ell$ | $\{0, 1, 2, 3\}$ | `Intn(4)` | Header obfuscation |
| XOR mask $\mu$ | $[0, 65535]$ | `Uint32()` | Length field masking |
| Decoy count $d$ | $[0, 3]$ | `Intn(4)` | Field count variation |
| Decoy sizes | $[1, 8]$ each | `Intn(8)+1` | Field size variation |
| Field order $\pi$ | $(5 + d)!$ perms | `Shuffle` | Structure randomization |
| Pad min $p_{\min}$ | $[0, 15]$ | `Intn(16)` | Minimum padding |
| Pad max $p_{\max}$ | $[p_{\min} + 1, p_{\min} + 64]$ | `Intn(64)+1` | Maximum padding |
| Wagon min $W_{\min}$ | $[\mathrm{MSS}{+}16, \mathrm{MSS}{+}31]$ | `MSS+16+Intn(16)` | Min wagon plaintext size |
| Wagon max $W_{\max}$ | $[W_{\min}{+}24, W_{\min}{+}103]$ | `Intn(80)+24` | Max wagon plaintext size |

### 4.4 Combinatorial Diversity

The total number of distinct genomes can be bounded as follows:

$$|G| = 256 \times 2 \times 4 \times \sum_{d=0}^{3} \left( \binom{4}{1}^{?} \cdot 8^d \cdot (5+d)! \cdot 16 \cdot 64 \right) \tag{6}$$

More precisely:

$$\begin{aligned} |G| &= 256 \cdot 2 \cdot 4 \cdot 16 \cdot 64 \cdot \sum_{d=0}^{3} 8^d \cdot (5+d)! \\ &= 2{,}097{,}152 \cdot (120 + 8 \cdot 720 + 64 \cdot 5040 + 512 \cdot 40320) \\ &= 2{,}097{,}152 \cdot (120 + 5{,}760 + 322{,}560 + 20{,}643{,}840) \\ &= 2{,}097{,}152 \cdot 20{,}972{,}280 \\ &\approx 4.4 \times 10^{13} \end{aligned} \tag{7}$$

This does not account for the XOR mask (an additional $2^{16}$ factor when XOR encoding is selected) or decoy XOR masks. The effective diversity exceeds $10^{13}$ distinct wire formats, making signature-based detection infeasible.

### 4.5 Wire Format

Each packet on the wire has the following structure:

| 0 1 2 3 4 5 6 7 | 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 |
|---|---|
| Magic | Header fields in permuted order $\pi$ |
| Header fields (continued, variable length) | |
| Ciphertext (variable length) | |
| Padding ($p_{\min}$ to $p_{\max}$ random bytes) | |

The header fields are serialized according to the permutation $\pi$ determined by the genome. An observer without the PSK cannot determine where one field ends and another begins, as field boundaries depend on the genome-derived order and sizes.

## 5 Cryptographic Layer

### 5.1 Authenticated Encryption

Chameleon supports three AEAD constructions [11]:

- **ChaCha20-Poly1305** [13]: 256-bit key, 96-bit nonce (the default).
- **XChaCha20-Poly1305**: 256-bit key, 192-bit nonce. Selected when the genome specifies $n_s = 24$.
- **AES-256-GCM**: 256-bit key, 96-bit nonce. Available as a configuration option for hardware-accelerated environments.

All three produce a 128-bit (16-byte) authentication tag. The tag is placed as a separate field in the genome-defined header (not appended to ciphertext as in standard AEAD usage), further obscuring the packet structure.

### 5.2 Epoch-Based Nonce Construction

Traditional AEAD usage generates random nonces or uses sequential counters appended to ciphertext. CHAMELEON constructs nonces from a genome-derived base and a monotonic epoch counter:

$$\text{Nonce}[0 \ldots n_s{-}5] = B_{\text{nonce}}[0 \ldots n_s{-}5], \quad \text{Nonce}[n_s{-}4 \ldots n_s{-}1] = \text{epoch} \tag{8}$$

where epoch is a 32-bit big-endian counter incremented with each packet. The epoch is also included in the AEAD associated data to bind the ciphertext to its position in the sequence.

This construction provides two guarantees:

1. **No nonce reuse**: The epoch is monotonically increasing. Combined with the genome-derived prefix, each nonce is unique within and across sessions (with overwhelming probability).
2. **No random nonce transmission**: The nonce base is derived from the PSK; only the 4-byte epoch varies per packet, saving bandwidth compared to transmitting full random nonces.

### 5.3 Anti-Replay Protection

The receiver maintains a sliding window bitmap of size $W = 256$ epochs. The check proceeds as:

**Algorithm 1:** Anti-replay check with sliding window.

```
Input: Received epoch e, current maximum e_max, bitmap B[0...W-1]
Output: Accept or reject
if e = 0 then
    return reject ;                                   // Epoch 0 is reserved
end
if e > e_max then
    Shift window: clear bits for [e_max + 1, e];
    e_max ← e;
    Mark B[e mod W];
    return accept;
end
if e_max − e ≥ W then
    Reset window ;                 // Client reconnected with new initial epoch
    return accept;
end
if B[e mod W] is set then
    return reject ;                                   // Duplicate packet
end
Mark B[e mod W];
return accept;
```

Each tunnel session begins with a random initial epoch (drawn from the upper half of the 32-bit range) to prevent replay window collisions when a client reconnects to a running server.

## 6 Reliable Stream Multiplexing

CHAMELEON multiplexes multiple bidirectional streams over a single UDP tunnel. The multiplexer provides reliable, ordered delivery on a per-stream basis, similar to TCP but over UDP.

### 6.1 Commands

Four command types are defined:

Table 2: Multiplexer command types.

| Code | Command | Fields |
|---|---|---|
| `0x01` | OPEN | StreamID (4B), Port (2B), AddrLen (2B), Addr (var) |
| `0x02` | DATA | StreamID (4B), Seq (4B), Payload (var) |
| `0x03` | CLOSE | StreamID (4B) |
| `0x04` | ACK | StreamID (4B), AckSeq (4B), Window (4B), NumSACK (2B), SACK blocks (8B each) |

Stream IDs use an odd/even allocation scheme: the client allocates odd IDs (1, 3, 5, ...) and the server allocates even IDs (2, 4, 6, ...). This eliminates the need for ID negotiation.

### 6.2 Selective Acknowledgment

Each ACK command carries:

- A *cumulative* acknowledgment number (all data up to this sequence number has been received).
- A *receive window* advertisement (bytes the receiver is willing to accept).
- Zero or more *SACK blocks*, each specifying a contiguous range $[\text{left}, \text{right})$ of received out-of-order data.

SACK blocks enable the sender to retransmit only the specific segments that were lost, rather than retransmitting everything from the cumulative ACK point forward.

### 6.3 RTT Estimation

Round-trip time is estimated using the Jacobson/Karels algorithm [9]:

$$\text{RTTVAR} \leftarrow \tfrac{3}{4}\,\text{RTTVAR} + \tfrac{1}{4}\,|\text{SRTT} - R| \tag{9}$$

$$\text{SRTT} \leftarrow \tfrac{7}{8}\,\text{SRTT} + \tfrac{1}{8}\,R \tag{10}$$

$$\text{RTO} = \text{SRTT} + 4 \cdot \text{RTTVAR}, \quad \text{clamped to } [200\text{ms}, 60\text{s}] \tag{11}$$

where $R$ is a new RTT sample. Karn's algorithm is applied: only segments that have not been retransmitted contribute to RTT samples, preventing ambiguous measurements from corrupting the estimator.

### 6.4 Congestion Control

The multiplexer implements NewReno congestion control [8] with the following parameters:

**Slow start.** CWND increases by the number of acknowledged bytes per ACK, approximately doubling per RTT.

**Congestion avoidance.** CWND increases by MSS $\times$ ackedBytes/CWND per ACK, achieving approximately one MSS increase per RTT (additive increase).

Table 3: Congestion control parameters.

| Parameter | Value | Rationale |
|---|---|---|
| Initial CWND | 256 KB | Avoid starving parallel streams in slow start |
| Minimum CWND | 32 KB | Prevent collapse after loss |
| Maximum CWND | 16 MB | Full utilization on fast links |
| MSS | 1,200 B | Fits within typical UDP MTU budget |

**Fast recovery.** Upon detecting three duplicate ACKs, the sender sets ssthresh = CWND/2, then CWND = ssthresh $+ 3 \times$ MSS. The sender retransmits the oldest unacknowledged segment and remains in recovery until an ACK advances the cumulative acknowledgment.

**Timeout.** If the RTO expires before an ACK arrives, ssthresh = CWND/2 and CWND = minCWND. The RTO is doubled (exponential backoff).

### 6.5 Per-Stream Flow Control

Each stream maintains an independent flow controller with a default receive window of 4 MB (maximum 64 MB). The receive window is advertised in every Ack command. Flow control is advisory: the receiver never drops data that exceeds the window, but the sender throttles its output when the advertised window is insufficient.

On the send side, each stream limits outstanding (unacknowledged) bytes to 256 KB. This serves as a send-side flow control that naturally paces UDP packets, preventing burst drops that occur when thousands of small packets are sent in rapid succession.

### 6.6 Delayed Acknowledgment

To reduce ACK overhead, the multiplexer sends ACKs every 32 received data packets rather than per-packet. A 200 ms timer ensures that pending ACKs are flushed even during low-activity periods. ACKs are also triggered immediately when the receive window drops below 25% of its default value.

## 7 Stream Mode: Constant-Rate Cover Traffic

### 7.1 Motivation: Timing and Activity Inference

Structural polymorphism (section 4) defeats signature-based DPI by making each packet statistically indistinguishable from random. However, a passive observer recording the *flow-level* pattern—packet arrival times and counts—can still infer *when* a user is active, even without decrypting a single byte. This ability enables website fingerprinting [4, 14], flow correlation across hops [12], and statistical traffic analysis [19]. Efficient countermeasures have historically been difficult to deploy without prohibitive cost [6].

A natural defense is *constant-rate cover traffic*: emit packets at a rate that is independent of application activity, so the observer sees no correlation between the encrypted flow and real events. Chameleon's stream mode implements this defense while remaining consistent with the polymorphic-per-session design. Instead of a single fixed rate, Chameleon ties all observable cover parameters—packet size, flow rate envelope, burst thresholds—to the session genome, so that an observer without the PSK cannot even enumerate the envelope used by a particular session.

## 7.2 Wagon Wire Format

When stream mode is enabled, the plaintext carried inside the AEAD seal of every packet is a *wagon*: a length-prefixed container of fixed size. The layout is:

| 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 | 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 |
|---|---|
| RealLen (uint16 BE) | Real bytes (up to RealLen) |
| Real bytes (continued) | |
| Filler (zero bytes to total wagon size $W$) | |

RealLen is a 16-bit big-endian length. A value of zero marks a pure-chaff wagon; the receiver silently discards the entire plaintext. A nonzero value names the number of real mux-command bytes immediately following; the filler tail is left as zeros because AEAD output is already indistinguishable from random on the wire, and the receiver never inspects the filler (it reads only RealLen and the first RealLen bytes that follow). This saves a `crypto/rand` syscall on every packet.

Both endpoints use the wagon layout symmetrically; stream mode is an opt-in setting on each side, and peers must agree.

## 7.3 Genome-Derived Wagon Sizes

For each outgoing packet the sender draws a wagon size $W$ uniformly from $[W_{\min}, W_{\max}]$, where $W_{\min}$ and $W_{\max}$ are the genome-derived bounds introduced in section 4.3. This gives three simultaneous properties:

1. **Per-packet variation.** Every wagon has a different plaintext size (up to ciphertext-length variation), so an observer cannot exploit a fixed-size signature or exact-size correlation across packets.
2. **Genome-bound envelope.** Both $W_{\min}$ and $W_{\max}$ are HKDF-derived from the PSK. Only a peer who knows the PSK can predict the envelope. An observer sees random sizes but cannot determine the underlying range.
3. **Asynchronous agreement.** The sender draws $W$ locally; the receiver does not need to predict it (the UDP datagram boundary carries the size). Packet loss or reordering does not desynchronize any shared PRNG state—there is none.

The invariant $W_{\min} \geq \mathrm{MSS} + 2$ guarantees that the largest mux command (a DATA segment with a 1200-byte payload plus 9-byte header) fits inside the smallest possible wagon with its 2-byte prefix. No mux-layer fragmentation is required.

## 7.4 Rate Envelope with Random Walk Drift

The emission rate is controlled by a single pump goroutine on each side, configured with operator-chosen bounds $R_{\min}$ and $R_{\max}$ in plaintext bytes per second. On a per-session basis the pump maintains a current target rate $R(t)$ that is nudged once per second by a reflected random walk:

$$R(t+1) = \mathrm{clamp}_{[R_{\min}, R_{\max}]}(R(t) + \Delta)\,, \quad \Delta \sim \mathcal{U}\Big(-\tfrac{R_{\max}-R_{\min}}{8},\ +\tfrac{R_{\max}-R_{\min}}{8}\Big) \tag{12}$$

Between drift updates the pump schedules its next emission at

$$t_{\mathrm{next}} = t_{\mathrm{prev}} + \frac{W}{R(t)}, \tag{13}$$

where $W$ is the wagon size chosen for the packet. When the envelope is wider than typical real-traffic rates, the flow rate appears to drift irregularly inside the envelope even under a steady load—described informally as "a river that is never at full flow". This non-trivial rate variation, together with genome-varied wagon sizes, complicates simple volume-versus-time fingerprinting.

Wagon size $W$ is drawn at every tick, so the *byte rate* follows $R(t)$ while the *packet rate* is slightly modulated by the wagon-size distribution.

### 7.5 Burst: Handling Real Data That Exceeds the Envelope

Constant-rate cover naturally conflicts with large downloads: if application bytes arrive faster than $R_{\max}$, something must give. The protocol's pump resolves this with a bounded *burst mode*.

The mux layer pushes encoded commands into a queue of capacity $Q = 1024$ wagons. The pump tracks the queue depth $q$ with hysteresis:

$$q \geq Q_{\text{high}} \Rightarrow \text{enter burst}, \qquad Q_{\text{high}} = 128 \tag{14}$$

$$q \leq Q_{\text{low}} \Rightarrow \text{exit burst}, \qquad Q_{\text{low}} = 32 \tag{15}$$

In burst mode the effective rate used in (13) is replaced by $R_{\text{burst}} = \beta \cdot R_{\max}$ with $\beta = 4$. This preserves a *hard cap* on the wire rate—essential to avoid overrunning kernel UDP buffers, which would otherwise manifest as packet loss and trigger mux retransmissions that can collapse the stream—while allowing throughput to rise above the cover envelope when real data demands it. Both the hysteresis and $\beta$ are empirically tuned; see section 9.6.

The queue capacity is large enough that mux backpressure (through blocking `Enqueue`) engages only after burst has been active long enough to make meaningful headway, avoiding an oscillation in which mux-layer write throttling and pump drain rate alternately dominate.

### 7.6 Pump Architecture and Single-Writer Model

With no pump attached, many mux goroutines may call `Tunnel.Send` concurrently; `*net.UDPConn` serializes writes at the socket level but packet order is otherwise unspecified. With a pump attached, a single pump goroutine is the sole writer to the UDP socket. This gives three desirable properties at no cost:

1. Strictly ordered emissions (the ordering on the wire follows the pump's timing, not the mux's producer threads).
2. No cross-goroutine interleaving of packets.
3. Cadence that tracks $R(t)$ with no jitter introduced by producer scheduling.

The pump holds a scratch buffer of $W_{\max}$ bytes that is reused across ticks; only the real-data `copy` is needed per packet, with zeroing of the filler tail via `clear`. A server pump also checks peer availability before emitting: until the server has discovered its first valid client packet (from which it learns the peer address) the pump preserves its cadence but emits no packets, avoiding log spam and wasted wire bytes.

### 7.7 Cover Properties

The combined effect of the above mechanisms is:

- In the *idle regime*, packets flow at rate $R(t) \in [R_{\min}, R_{\max}]$ with genome-varying sizes; content is pure chaff but encrypted and indistinguishable from real traffic.
- In the *partial-load regime* (aggregate real data below $R_{\max}$), real bytes replace filler within the same envelope; the wire rate is identical to the idle regime.
- In the *full-load regime* (real data above $R_{\max}$), burst lifts the rate up to $\beta R_{\max}$, wagons are packed to the brim with real bytes, and filler shrinks toward zero. The rate increase reveals *that* the user is pushing data but not what or where.

Narrowing the envelope (e.g., setting $R_{\min} = R_{\max} = 100$ KB/s) gives strict cover traffic at the cost of bounded throughput; widening it (e.g., 500 KB/s–3 MB/s, the default) balances practical usability with continued indistinguishability of browsing versus idle activity. The operator chooses the tradeoff.

# 8 Security Analysis

## 8.1 Threat Model

We consider three adversary types:

1. **Passive observer with DPI**: Can observe all packets between client and server, perform statistical analysis, and match against known protocol signatures.
2. **Active prober**: Can send crafted packets to the server to elicit recognizable responses.
3. **Replay attacker**: Can record and retransmit previously observed packets.

We assume the adversary does not possess the PSK.

## 8.2 Resistance to Passive Analysis

**Structural fingerprinting.** Traditional DPI matches fixed byte patterns (e.g., TLS record type at byte 0, HTTP method at bytes 0–3). In Chameleon, the magic byte, field order, and field sizes change per session. The probability that two sessions share the same genome is at most $|G|^{-1} \approx 2.3 \times 10^{-14}$ (eq. (7)), making it infeasible to build a structural signature.

**Entropy analysis.** Each packet's output is designed to be indistinguishable from uniform random data. The encrypted payload (ChaCha20-Poly1305 or AES-GCM output) has near-maximal entropy by construction. Padding bytes are generated from `crypto/rand`. Decoy fields are random bytes XORed with genome-derived masks, producing uniformly distributed output.

We measure Shannon entropy $H$ over the byte distribution of wire output:

$$H = -\sum_{i=0}^{255} p_i \log_2 p_i \tag{16}$$

Empirical measurements on captured traffic yield $H \geq 7.9$ bits/byte (maximum is 8.0 for uniform distribution), confirmed by the entropy test in the test suite (`TestFrameEntropy`).

**Length analysis.** Variable-length padding ($[p_{\min}, p_{\max}]$ bytes per packet) masks payload lengths. The four different length encoding schemes further complicate length-based correlation across sessions.

## 8.3 Resistance to Active Probing

The server processes a received packet through the following pipeline:

1. Magic byte check (immediate discard on mismatch).
2. Morph decode (fails without correct genome).
3. Anti-replay check (rejects duplicate/out-of-window epochs).
4. AEAD decryption (fails without correct key).

An active prober without the PSK fails at step 1 with probability $255/256 \approx 99.6\%$ and at step 4 with certainty (AEAD forgery is computationally infeasible). The server never sends a response to invalid packets, revealing no information to the prober.

## 8.4 Replay Protection

The 256-epoch sliding window (algorithm 1) rejects duplicate packets. The random initial epoch prevents an attacker from precomputing a valid epoch sequence. Even if an attacker replays a valid packet from a previous session, it will fail AEAD decryption because the session keys (derived from the same PSK but potentially different salt) differ.

### 8.5 Resistance to Timing and Activity Inference

Without stream mode, CHAMELEON inherits the flow-level fingerprinting weakness common to all randomization-based tunnels: even with uniformly random packet content, *when* a packet is sent is correlated with real application activity. Website fingerprinting attacks [4, 14] and flow correlation attacks [12] exploit precisely this signal.

Stream mode (section 7) attacks this signal directly:

1. In the idle and partial-load regimes, the wire rate is completely independent of application activity. An observer cannot distinguish a user browsing a news site from a user with no real traffic at all.
2. In the full-load regime, the observer learns that the user is producing bytes at a rate above $R_{\max}$, but not the content, destination, or flow boundaries. This is analogous to observing a VPN user's aggregate bandwidth without learning what sites they visit.
3. Per-packet sizes are drawn from the genome-derived envelope $[W_{\min}, W_{\max}]$; only a peer with the PSK can predict the range. Size-based attacks such as the ones in [6] require stable size distributions, which CHAMELEON denies by tying them to the session PSK.

Operators seeking stricter cover can narrow the envelope, at the cost of throughput. The protocol does not attempt to hide that *some* cover traffic is flowing—that would require a complete blending attack against a host protocol—but it hides *when* the user is active.

### 8.6 Known Limitations

1. **High-entropy detection.** An adversary who blocks *all* high-entropy traffic (the "noise paradox") would also block CHAMELEON. This is a fundamental limitation of any randomization-based approach.
2. **No forward secrecy.** Without an ephemeral key exchange (e.g., ECDH), compromise of the PSK allows decryption of all past and future sessions.
3. **Cover breach during burst.** When real traffic exceeds the envelope, the pump's burst mode lifts the wire rate above $R_{\max}$, visibly signaling heavy activity. Narrow envelopes trade throughput for stricter cover.
4. **Cover cost.** Stream mode continuously emits bytes even when the user is idle. At the default 500 KB/s–3 MB/s envelope this consumes non-trivial mobile bandwidth. The mode is opt-in for this reason.

## 9 Performance Evaluation

### 9.1 Test Environment

We evaluate CHAMELEON on a real-world link between a client in Russia and a VDS (Virtual Dedicated Server) in the Netherlands. The baseline (direct) connection provides approximately 100 Mbit/s.

### 9.2 Throughput

The discrepancy between file transfer throughput (30 Mbit/s) and Speedtest.net results (2.16 Mbit/s) is attributable to Ookla's measurement methodology, which uses many short-lived TCP connections that do not allow the congestion window to fully open.

### 9.3 Concurrency

Loading speedtest.net through the tunnel generates 67 parallel resource requests. All 67 streams complete successfully with zero pending or stalled streams. Five consecutive runs of a 6.8 MB

Table 4: Throughput and latency measurements.

| Metric | Value |
|---|---|
| Peak throughput | 30 Mbit/s (3.75 MB/s) |
| Sustained throughput (6.8 MB file, 5 runs) | 30 Mbit/s, $\sigma < 1$ Mbit/s |
| Speedtest.net (Ookla methodology) | 2.16 / 1.17 Mbit/s |
| Simple site latency (example.com) | 150–500 ms |
| Complex site latency (Wikipedia) | ∼400 ms |
| Parallel 3-site load | ∼800 ms |

file download show no throughput degradation between runs, confirming that the congestion control correctly recovers window state.

### 9.4 Overhead

Per-packet overhead consists of:

- Morph header: magic byte (1B) + header fields (variable, typically 25–50B depending on genome)
- AEAD tag: 16B
- Padding: $[p_{\min}, p_{\max}]$ bytes (typically 0–79B)

Total overhead ranges from approximately 100 to 150 bytes per packet, representing 8–12% of a 1,280-byte UDP payload.

### 9.5 Automated Browser Testing

We validate real-world compatibility using Playwright-based browser automation through the SOCKS5 proxy. All 12 test scenarios pass:

Table 5: Playwright browser test results.

| Test | Description | Time |
|---|---|---|
| Simple site load | example.com, httpbin.org | 150–500 ms |
| Complex site load | Wikipedia (CSS/JS/images) | ∼400 ms |
| Parallel multi-site | 3 sites simultaneously | ∼800 ms |
| Resource-heavy site | speedtest.net (67 resources) | Pass |

### 9.6 Stream Mode Throughput

We evaluate the overhead of stream mode across rate envelopes using two harnesses. The first is `cmd/bench-stream`, an in-process benchmark that sets up both endpoints on a single Windows host over UDP loopback and drives 10 MB through the mux layer. The second deploys the server on the VDS, runs the client on the same host (loopback), and downloads a 10 MB file from `proof.ovh.net`; the direct path (no tunnel) from the VDS to OVH caps at 608 KB/s on the evaluated link.

In both harnesses the stream mode preserves the bulk of the native throughput once the pump parameters are tuned. The idle (chaff-only) mode holds the configured envelope—an `-idle` subcommand of `bench-stream` measures the wire rate and reports it stays inside $[R_{\min}, R_{\max}]$.

Two factors motivated the tuning. (i) The original queue capacity of 128 wagons was small enough that mux `Write` blocked on `Enqueue` before the queue ever reached the burst threshold; the pump then oscillated at the envelope midpoint instead of engaging burst. Raising the cap to

Table 6: Stream-mode throughput on Windows UDP loopback before and after the pump-tuning optimizations (queue cap 128→1024, hysteresis burst, $\beta = 4$ rate cap, `fillWagon` scratch reuse). 10 MB sent.

| **Envelope (bytes/sec)** | **Before** | **After** |
|---|---:|---:|
| [100,000, 700,000] | 0.18 MB/s | 2.43 MB/s |
| [500,000, 3,000,000] | 0.81 MB/s | 10.4 MB/s |
| [1,000,000, 5,000,000] | (stream aborted) | 17.4 MB/s |
| [2,000,000, 10,000,000] | (stream aborted) | 34.6 MB/s |

Table 7: Stream-mode goodput over the VDS (Netherlands) to `proof.ovh.net`, 10 MB. Direct path (no tunnel) measures 608 KB/s.

| **Configuration** | **Throughput** | **vs. direct** |
|---|---:|---:|
| Direct (no tunnel) | 608 KB/s | 100% |
| Stream [100,000, 700,000] | 524 KB/s | 86% |
| Stream [500,000, 3,000,000] (default) | 523 KB/s | 86% |
| Stream [1,000,000, 5,000,000] | 590 KB/s | 97% |
| Stream [2,000,000, 10,000,000] | 560 KB/s | 92% |

1024 with a $Q_{\text{high}} = 128, Q_{\text{low}} = 32$ hysteresis restored burst engagement without the flapping. (ii) An initial implementation removed the rate cap entirely during burst, which on loopback overran the kernel UDP buffer and caused mux retransmissions to exhaust their budget. The $\beta = 4$ cap empirically maximizes burst throughput while keeping the socket below its drop point. These two changes are the difference between the "before" and "after" columns of table 6.

## 10 Future Work

**Forward secrecy via ECDH.** Integrating an ephemeral Curve25519 key exchange with Elligator 2 encoding [2, 3] would provide forward secrecy while maintaining the indistinguishability property. The exchanged ephemeral secret would serve as the HKDF salt, causing the genome to change per-session even with a fixed PSK.

**BBR congestion control.** The current NewReno implementation can underperform on high-bandwidth, high-latency links. Implementing BBR (Bottleneck Bandwidth and Round-trip propagation time) would improve throughput on such paths. The multiplexer's `CongestionController` interface supports drop-in replacement.

**Protocol mimicry.** An optional mode could prepend TLS Client Hello or HTTP headers to the polymorphic payload, satisfying allowlist-based DPI that requires recognized protocol preambles. This would be controlled by a genome flag.

**UDP ASSOCIATE.** Extending the SOCKS5 implementation with UDP ASSOCIATE (command `0x03`) would support tunneling of UDP-based applications such as DNS and VoIP.

## 11 Conclusion

We have presented Chameleon, a polymorphic UDP tunneling protocol that derives a unique wire format for each session from a pre-shared key. The genome derivation mechanism uses

HKDF-SHA256 for key expansion and a deterministic SHA-256 counter-mode PRNG to generate session-specific parameters: magic byte, header field permutation, decoy fields, nonce size, length encoding scheme, padding range, and wagon size range. This yields over $10^{13}$ possible wire formats, making signature-based DPI detection infeasible.

The protocol achieves 7.9 bits/byte Shannon entropy on the wire, provides authenticated encryption with epoch-based nonce construction, and resists replay attacks through a 256-epoch sliding window. A stream multiplexer with SACK, NewReno congestion control, and per-stream flow control delivers reliable, ordered data over UDP at 30 Mbit/s with 67 concurrent streams. An optional stream mode layers a constant flow of genome-shaped wagons on top of the transport: real data and random chaff share the same wire envelope, so a passive observer cannot tell when the user is active. Empirically, the mode preserves 86–97% of direct throughput at a 500 KB/s–3 MB/s envelope, with burst thresholds tuned to avoid UDP overrun.

Chameleon demonstrates that per-session wire-format polymorphism is a viable and efficient approach to circumventing deep packet inspection. Combined with genome-shaped cover traffic, it offers stronger fingerprint and timing-analysis resistance than fixed-format randomization while maintaining practical throughput for real-world use.

The implementation is released under the GNU General Public License v3.0 (GPL-3.0) and is available at https://github.com/william-aqn/genome. This paper is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0).

## References


[1] Yawning Angel. obfs4 (the obfourscator). Technical report, Tor Project, 2014. https://gitlab.com/yawning/obfs4/-/blob/master/doc/obfs4-spec.txt.

[2] Daniel J. Bernstein. Curve25519: New Diffie-Hellman speed records. In *Public Key Cryptography – PKC 2006*, pages 207–228, 2006.

[3] Daniel J. Bernstein, Mike Hamburg, Anna Krasnova, and Tanja Lange. Elligator: Elliptic-curve points indistinguishable from uniform random strings. In *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security*, pages 967–980, 2013.

[4] Xiang Cai, Xin Cheng Zhang, Brijesh Joshi, and Rob Johnson. Touching from a distance: Website fingerprinting attacks and defenses. In *Proceedings of the 2012 ACM Conference on Computer and Communications Security (CCS)*, pages 605–616, 2012.

[5] Roger Dingledine, Nick Mathewson, and Paul Syverson. Tor: The second-generation onion router. In *Proceedings of the 13th USENIX Security Symposium*, pages 303–320, 2004.

[6] Kevin P. Dyer, Scott E. Coull, Thomas Ristenpart, and Thomas Shrimpton. Peek-a-boo, I still see you: Why efficient traffic analysis countermeasures fail. In *Proceedings of the 2012 IEEE Symposium on Security and Privacy*, pages 332–346, 2012.

[7] Ronald A. Fisher and Frank Yates. *Statistical Tables for Biological, Agricultural and Medical Research.* Oliver and Boyd, Edinburgh, 1938.

[8] T. Henderson, S. Floyd, A. Gurtov, and Y. Nishida. The NewReno modification to TCP's fast recovery algorithm. RFC 6582, IETF, April 2012.

[9] Van Jacobson. Congestion avoidance and control. In *Proceedings of ACM SIGCOMM*, pages 314–329, 1988.

[10] H. Krawczyk and P. Eronen. HMAC-based extract-and-expand key derivation function (HKDF). RFC 5869, IETF, May 2010.

[11] D. McGrew. An interface and algorithms for authenticated encryption. RFC 5116, IETF, January 2008.

[12] Milad Nasr, Alireza Bahramali, and Amir Houmansadr. DeepCorr: Strong flow correlation attacks on Tor using deep learning. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1962–1976, 2018.

[13] Y. Nir and A. Langley. ChaCha20 and Poly1305 for IETF protocols. RFC 8439, IETF, June 2018.

[14] Andriy Panchenko, Fabian Lanze, Jan Pennekamp, Thomas Engel, Andreas Zinnen, Martin Henze, and Klaus Wehrle. Website fingerprinting at Internet scale. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2016. https://www.ndss-symposium.org/wp-content/uploads/2017/09/website-fingerprinting-internet-scale.pdf.

[15] Shadowsocks Contributors. Shadowsocks: A fast tunnel proxy, 2012. https://shadowsocks.org.

[16] V2Ray Contributors. VMess protocol specification, 2016. https://www.v2fly.org/en_US/developer/protocols/vmess.html.

[17] Qiyan Wang, Xun Gong, Giang T.K. Nguyen, Amir Houmansadr, and Nikita Borisov. CensorSpoofer: Asymmetric communication using IP spoofing for censorship-resistant web browsing. In *Proceedings of the 2012 ACM Conference on Computer and Communications Security*, pages 121–132, 2012.

[18] Philipp Winter, Tobias Pulls, and Juergen Fuss. ScrambleSuit: A polymorphic network protocol to circumvent censorship. In *Proceedings of the 12th ACM Workshop on Privacy in the Electronic Society (WPES)*, pages 213–224, 2013.

[19] Charles V. Wright, Scott E. Coull, and Fabian Monrose. Traffic morphing: An efficient defense against statistical traffic analysis. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2009. https://www.ndss-symposium.org/ndss2009/traffic-morphing-efficient-defense-against-statistical-traffic-analysis/.